Studio Nx

JS 292 — MIDI 노트 재생과 악기 층

2026-10-19

JavaScript 292 — MIDI 노트 재생과 악기 층 (`audio/notes.js`, `audio/instruments.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/notes.js`, `studionx/frontend/static/js/audio/instruments.js`
작업	S1-11a 재생 오디오 그래프: MIDI 클립의 노트를 트랙의 악기 층으로 연주 (FR-205, FR-233, FR-241)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 291 Synth, 다음 293 이펙트

1. 하는 일

두 파일이 함께 노트를 소리로 바꿉니다. `notes.js`는 "어느 노트가 언제 시작하는지"를 계산하고, `instruments.js`는 "그 노트를 트랙의 어떤 악기로 연주하는지"를 맡습니다.

```
MIDI 클립 { start_tick, length_tick, offset_tick, notes[] }
      │  expandNotes        노트를 타임라인 절대 위치로 펼칩니다 (음소거 제외, 클립 끝에서 자름)
      ▼
notesStartingIn(구간)      이번 단계에서 새로 시작하는 노트를 고릅니다
      ▼
planNote                   when(시각), durationSec(길이)로 바꿉니다
      ▼
NotePlayer.schedule ─▶ InstrumentRack.playNote ─▶ 트랙의 악기 층마다 playSynthNote
                                                  (층: 볼륨, 팬 노드 ─▶ 트랙 입구)
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession.start`	재생을 시작할 때 MIDI 클립 전체를 `expandNotes`로 한 번 펼쳐 둡니다.
`PlaybackSession._onStep`	단계마다 `notesStartingIn`으로 새 노트를 골라 `NotePlayer.schedule`로 예약합니다.
정지, 일시정지, 루프 되감기	`NotePlayer.stopAll`이 울리는 노트를 모두 끕니다.

3. 코드

3.1 노트 계산 (`notes.js`)

```
export function expandNotes(clips) {
  const out = [];
  (clips || []).forEach(function (clip) {
    const offset = clip.offset_tick || 0;
    const clipEnd = clip.start_tick + clip.length_tick;
    (clip.notes || []).forEach(function (n) {
      if (n.is_muted) return;
      if (n.start_tick < offset) return;
      const abs = clip.start_tick + (n.start_tick - offset);
```

```
      if (abs >= clipEnd) return;
      out.push({
        track_id: clip.track_id,
        pitch: n.pitch,
        velocity: n.velocity != null ? n.velocity : 100,
        abs_start: abs,
        length_tick: Math.max(1, Math.min(n.length_tick, clipEnd - abs)),
      });
    });
  });
  out.sort(function (a, b) { return a.abs_start - b.abs_start; });
  return out;
}
```

```
export function notesStartingIn(events, fromTick, toTick) {
  return (events || []).filter(function (e) { return e.abs_start >= fromTick && e.abs_start < toTick;
});
}
```

```
export function planNote(ev, nowTick, nowTime, baseBpm, changes) {
  const sec = function (a, b) {
    return ticksToSecondsWithChanges(b, baseBpm, changes) - ticksToSecondsWithChanges(a, baseBpm, change
s);
  };
  return { when: nowTime + sec(nowTick, ev.abs_start), durationSec: sec(ev.abs_start, ev.abs_start + ev.
length_tick) };
}
```

3.2 노트 재생기 (`notes.js`)

```
schedule(ev, nowTick, nowTime, baseBpm, changes) {
  const plan = planNote(ev, nowTick, nowTime, baseBpm, changes);
  if (!(plan.durationSec > 0)) return 0;
  const voices = this.playOnTrack(ev.track_id, ev.pitch, ev.velocity, Math.max(plan.when, this.context.c
urrentTime), plan.durationSec);
  const self = this;
  (voices || []).forEach(function (v) {
    self.active.push({ voice: v, end: plan.when + plan.durationSec + 6 });   // 릴리스 여유 포함
  });
  this._prune();
  return (voices || []).length;
}
```

```
stopAll(atTime) {
  const t = atTime != null ? atTime : this.context.currentTime;
  this.active.forEach(function (a) { try { a.voice.stop(t); } catch (e) { /* 무시 */ } });
  this.active = [];
}
```

3.3 악기 층 (`instruments.js`)

```
_layer(trackId, inst) {
  const key = String(trackId) + ':' + String(inst.id);
```

```
  const ctx = this.context;
  let layer = this.layers[key];
  if (!layer) {
    const gain = createStereoGain(ctx);
    const pan = typeof ctx.createStereoPanner === 'function' ? ctx.createStereoPanner() : null;
    if (pan) { gain.connect(pan); pan.connect(this.trackInput(trackId)); }
    else gain.connect(this.trackInput(trackId));
    layer = { gain: gain, pan: pan };
    this.layers[key] = layer;
  }
  layer.gain.gain.value = layerGain(inst.volume_db);
  if (layer.pan) layer.pan.pan.value = Math.max(-1, Math.min(1, Number(inst.pan) || 0));
  return layer;
}
```

```
playNote(trackId, pitch, velocity, when, durationSec) {
  const out = [];
  const self = this;
  (this.getInstruments(trackId) || []).forEach(function (inst) {
    if (inst.is_enabled === false) return;
    if (PLAYABLE_CODES.indexOf(inst.plugin_code) < 0) return;
    const layer = self._layer(trackId, inst);
    const params = resolveParams(inst.schema, inst.params);
    out.push(playSynthNote(self.context, layer.gain, params, pitch, velocity, when, durationSec));
  });
  return out;
}
```

4. 최신 문법으로 쓰면

```
// expandNotes 를 구조 분해와 flatMap 으로 쓴 모양
export const expandNotes = (clips = []) =>
  clips
    .flatMap(({ track_id, start_tick, length_tick, offset_tick = 0, notes = [] }) =>
      notes
        .filter((n) => !n.is_muted && n.start_tick >= offset_tick
          && start_tick + (n.start_tick - offset_tick) < start_tick + length_tick)
        .map((n) => {
          const abs = start_tick + (n.start_tick - offset_tick);
          return {
            track_id,
            pitch: n.pitch,
            velocity: n.velocity ?? 100,
            abs_start: abs,
            length_tick: Math.max(1, Math.min(n.length_tick, start_tick + length_tick - abs)),
          };
        }))
    .sort((a, b) => a.abs_start - b.abs_start);
```

전통 문법 (실제 코드)	최신 문법
`forEach` 안에서 배열에 `push`	`flatMap`, `filter`, `map`의 연결
`n.velocity != null ? n.velocity : 100`	`n.velocity ?? 100`

전통 문법 (실제 코드)	최신 문법
`clip.offset_tick || 0`	구조 분해 기본값 `offset_tick = 0`

5. 왜 이렇게

선택	이유
재생 시작 때 한 번만 펼칩니다	노트는 수천 개일 수 있어서, 단계마다(1/16박) 전부 훑으면 느립니다. `abs_start` 순으로 정렬해 두고 구간만 걸러 냅니다.
클립의 `offset_tick`을 오디오 클립과 같은 뜻으로 씁니다	클립을 왼쪽에서 잘라 내면 `offset_tick`이 늘어납니다. 그 앞의 노트는 빼고, 남은 노트는 클립 시작에서 당겨 놓습니다.
클립 밖으로 나가는 노트는 클립 끝에서 자릅니다	클립을 줄였는데 노트가 클립 밖까지 울리면 안 됩니다.
음소거된 노트는 펼칠 때 뺍니다	재생 중에는 걸러 낼 필요가 없어서 가볍습니다.
악기 층마다 볼륨과 팬을 따로 둡니다	한 트랙에 악기가 여러 개일 때(레이어) 층마다 소리 크기와 좌우가 다릅니다. 층 노드는 한 번 만들어 재사용합니다.
Sampler는 MIDI로 소리가 나지 않습니다	시드의 Sampler에는 샘플을 고르는 칸이 없어서, 노트를 연주할 소리를 알 수 없습니다. 목업을 본 뒤 결정하기로 했습니다 (`PLAYABLE_CODES`에 `synth_sub`만 있음).
울리는 소리 목록을 정리합니다 (`_prune`)	끝난 소리를 계속 담아 두면 목록이 끝없이 자랍니다. 6초 지난 것은 뺍니다.
노트가 시작된 뒤에 재생을 시작하면 그 노트는 울리지 않습니다	단순함을 우선했습니다. 필요하면 시작 위치에 걸친 노트를 중간부터 울리게 확장할 수 있습니다.

6. 시험

- `docs/handoff/s1-11a_tests/synth_notes_unit.mjs`는 다음을 확인합니다. 음소거 노트는 빠지고 클립 밖 노트도 빠집니다. 클립 끝에서 길이가 잘리고, `velocity`가 없으면 100이 됩니다. `offset_tick`이 있으면 앞 노트가 빠지고 나머지가 당겨집니다. 구간 선택은 시작은 포함하고 끝은 포함하지 않습니다. 120BPM에서 960틱이 0.5초입니다.
- `docs/handoff/s1-11a_tests/ui_midi.mjs`는 실제 화면에서 MIDI 클립의 노트가 Synth 소리로 나오는지 확인합니다.